

Introduction

Every so often LeetCode gives you a problem that looks like a keypad trivia question and is actually a greedy algorithm in disguise. Minimum Number of Pushes to Type Word II is one of those. It's tagged easy, but it tests something interviewers genuinely care about: can you recognize a greedy structure, prove it's correct instead of just guessing, and turn that proof into six lines of clean code?

This problem is worth learning not because you'll ever redesign a phone keypad, but because the underlying pattern — sort by frequency, hand out cheap resources first — shows up constantly. Huffman coding, cache eviction, load balancing, even how your music app pre-loads the next song you're likely to play, all lean on the same idea. Once you see it here, you'll start spotting it everywhere.

Problem Overview

You're given a word, and you need to design a keypad from scratch to type it with as few button presses as possible.

The rules:

- You have 8 keys available, corresponding to keys 2 through 9 on a phone.
- You can assign any letters to any key, in any order — there's no fixed mapping like the classic "abc on key 2."
- Each key can hold as many letters as you want.
- If a letter is the n th letter placed on its key, pressing that key n times types that letter. So the first letter on a key costs 1 press, the second costs 2, the third costs 3, and so on.
- You must press buttons one at a time, in order, to spell the word — count every single press.
- The word can be up to 100,000 characters, but since English has only 26 letters, you're never dealing with more than 26 distinct characters.

Your job: choose the letter-to-key assignment that minimizes total presses for the given word.

Example

Take `word = "abcde"`. Every letter appears exactly once — no repeats. Since we control the keypad, we can give each letter its own key, so each one costs just 1 press. Total: 5 presses for 5 letters.

Now a harder one: word = "aabbcc". Here a appears 2 times, b appears 2 times, c appears 1 time. We have only 3 distinct letters and 8 keys, so every letter can sit alone in the cheapest slot on its own key, costing 1 press each. Total cost = $(2 \times 1) + (2 \times 1) + (1 \times 1) = 5$.

Intuition

Forget the phone for a second. Think about your spice rack. The sauce you reach for every single day sits right at the front — zero effort to grab. The jar you touch twice a year gets shoved on the top shelf, where reaching it costs you something. You're not being lazy, you're being efficient: you're paying the smallest cost for the thing you do most often.

That's the entire problem. Letters that show up a lot in the word should sit in the cheapest key slots (cost 1). Letters that barely show up can afford to sit in the expensive slots (cost 2, 3, 4) because you'll rarely pay that cost.

Here's the trap: your brain wants to picture a real phone keypad, where key 2 always means a, b, c. That layout was chosen for finger reach on a physical device, not for minimizing presses on a specific word. This problem explicitly lets you invent the layout, so if your word uses z constantly and a almost never, you're free to put z in a cheap slot and a in an expensive one. Reaching for the real keypad picture here will lead you straight to a wrong answer.

Why does sorting by frequency work? Let's actually check it instead of trusting our gut. Suppose a letter appearing 10 times is stuck in a cost-2 slot, while a letter appearing only 2 times sits in a cost-1 slot. Cost before swapping: $(10 \times 2) + (2 \times 1) = 22$. Now swap them — put the 10 in the cheap slot, the 2 in the expensive one: $(10 \times 1) + (2 \times 2) = 14$. The total dropped. Any time a more frequent letter sits in a more expensive slot than a less frequent one, swapping them helps or does nothing — never hurts. Repeat that swap until nothing is out of order, and you land on the fully sorted arrangement. That's an exchange argument, and it's the standard way to prove a greedy strategy is optimal.

Brute Force Solution

Idea: Try every possible way of partitioning the 26 letters across 8 keys and every possible ordering within each key, then compute the total cost for each arrangement and keep the minimum.

Algorithm: Generate all groupings of distinct letters into up to 8 non-empty ordered sequences, compute cost for each, take the best.

Advantages: Guaranteed correct by exhaustive search; no need to reason about why an arrangement is optimal.

Disadvantages: The number of ways to partition and order letters across 8 keys explodes combinatorially — this is nowhere close to feasible even for a modest alphabet, let alone real-time use.

Complexity: Exponential (or worse) time, since you're enumerating an astronomical number of partitions/orderings. Not viable for a problem where the answer needs to compute instantly.

Nobody actually codes the brute force here — it's useful only as a thought experiment to motivate why we need the greedy insight.

Optimal Solution

Once you trust the exchange argument, the algorithm is short:

1. **Count** how often each letter appears in the word.
2. **Sort** those counts from most frequent to least frequent.
3. **Assign ranks:** the most frequent letter is rank 0, next is rank 1, and so on (0-indexed).
4. **Compute cost per rank** using $\text{rank} // 8 + 1$. Ranks 0–7 cost 1 press (first letter on each of the 8 keys), ranks 8–15 cost 2 presses (second letter on each key), ranks 16–23 cost 3 presses, and ranks 24–25 (if all 26 letters appear) cost 4 presses.
5. **Sum** $\text{frequency} \times \text{cost}$ across all letters.

Think of it as a ladder with rungs of 8 letters each. Every time you climb a rung, the cost per letter goes up by exactly one.

Rank range Cost per press

0–7	1
8–15	2
16–23	3
24–25	4

Worked example: Suppose sorted frequencies are 5, 4, 4, 3, 3, 2, 2, 1, 1 (9 distinct letters). The first 8 letters each get their own key at cost 1: $5+4+4+3+3+2+2+1 = 24$. The 9th letter (frequency 1) is the second letter on some key, costing 2: $1 \times 2 = 2$. Total: $24 + 2 = 26$.

Python Code

```
from collections import Counter
```

```
def minimum_pushes(word: str) -> int:
    freq = Counter(word)
    counts = sorted(freq.values(), reverse=True)

    total = 0
    for i, count in enumerate(counts):
        cost = i // 8 + 1
        total += count * cost

    return total
```

Code Walkthrough

- `Counter(word)` scans the word once and returns how many times each letter appears — `Counter("aab")` gives `{'a': 2, 'b': 1}`.
- `freq.values()` pulls out just the frequency numbers; we no longer care which letter had which count.
- `sorted(..., reverse=True)` orders those counts from biggest to smallest, so the most common letters line up first — exactly what the Frequency Ladder needs.
- `enumerate(counts)` gives us both the position `i` (0-indexed rank) and the value `count` at each step, so we don't have to track position manually.
- `i // 8 + 1` converts rank into cost using integer division: ranks 0–7 give `0 // 8 + 1 = 1`, ranks 8–15 give `1 // 8 + ... = 2`, and so on — each crossing of a multiple of 8 bumps the cost by one.
- `total += count * cost` accumulates frequency times its assigned cost.

Dry Run

Trace `word = "aabbc"`.

`Counter("aabbc")` → `{'a': 2, 'b': 2, 'c': 1}` `sorted(freq.values(), reverse=True)`
→ `[2, 2, 1]`

i count cost = i//8+1 contribution

0	2	1	2
1	2	1	2
2	1	1	1

Total = `2 + 2 + 1 = 5`, matching the hand calculation from the example section.

Complexity Analysis

Time: $O(n)$, where n is the word length. Building the `Counter` requires one pass over the word — that's $O(n)$. The sort only ever operates on at most 26 numbers (one per distinct English letter), which is a fixed-size, essentially constant-time operation regardless of how long the word gets. So the whole thing scales linearly with the word length and stays fast even at the 100,000-character limit.

Space: $O(1)$ relative to the alphabet, since `Counter` and the sorted list hold at most 26 entries no matter how long the word is.

Alternative Solutions

You could use a `max-heap` (via `heapq` with negated counts) to repeatedly pull the most frequent remaining letter instead of sorting up front. It produces the same result but adds $O(\log k)$ overhead per extraction for no benefit here, since a plain sort of at most 26 elements is already about as cheap as computation gets. Sorting is the simpler, faster choice for this problem's constraints.

Edge Cases

- **Empty word:** no letters to count, loop never runs, total is 0.
- **Single repeated character** ("aaaa"): one distinct letter, rank 0, cost 1, total equals the word's length.
- **All 26 letters present:** the two least frequent letters land on ranks 24 and 25, both costing 4 presses — the maximum this problem allows.
- **Single character** ("a"): total is simply 1.
- **All letters equally frequent:** order among tied frequencies doesn't matter for the total — any tie-breaking still lands on the same overall sum since tied letters occupy equivalent-cost slots.

Common Mistakes

1. **Sorting ascending instead of descending.** Forgetting `reverse=True` puts your most frequent letter in the most expensive slot — exactly backwards, producing an inflated answer.
2. **Assuming the real phone keypad layout applies.** The problem explicitly lets you choose the layout; hardcoding `a`, `b`, `c` on key 2 ignores the entire point of the problem.
3. **Off-by-one errors in the cost formula.** Using `i // 8` without the `+ 1` gives the first letter cost 0, which is impossible — every press costs at least 1.

4. **Forgetting only distinct letters matter for ranking.** Sorting the raw word instead of the frequency counts wastes effort and complicates the logic unnecessarily.
5. **Overcomplicating with a heap or priority queue.** It works, but for a fixed list of at most 26 numbers, a simple sort is simpler to write, easier to verify, and just as fast in practice.

Interview Questions

- Can you prove that sorting by frequency is optimal, not just intuitively reasonable?
- What changes if there were only 4 usable keys instead of 8?
- How would you handle the case where letters must keep their relative first-appearance order instead of being freely assigned (this is the "Part I" variant of the problem)?
- What's the time complexity if the word length were unbounded but the alphabet were much larger, say Unicode?
- Could you solve this with a max-heap instead of sorting, and what would the tradeoffs be?

Similar Problems

- **Minimum Number of Pushes to Type Word I** — same idea, but letters must be assigned keys in the order they first appear in the word, not by frequency. A good next step to test whether you actually understood the greedy reasoning here or just memorized the code.
- **Reorganize String** — another greedy problem built around sorting by frequency and placing the most common elements carefully.
- **Huffman Coding (conceptual)** — the classic algorithm this problem echoes: give frequent symbols short codes, rare symbols long codes.
- **Top K Frequent Elements** — shares the "count then sort/select by frequency" backbone.

Key Takeaways

Sort by frequency, hand out the cheapest resources to the most common items, and prove greedy correctness with a simple swap argument instead of hoping it's right. The `rank // 8 + 1` formula is just a compact way of expressing "every full round through your available keys costs one more press." Four small Python tools — `Counter`, `sorted(reverse=True)`, `enumerate`, and `//` — are all it takes to turn a clean piece of reasoning into a correct, fast solution.

Watch the Video

For the full walkthrough with the hand-traced examples and the swap-argument proof worked out step by step, watch the video here: {LINK}

About the Series

This is part of the Fun with Learning Technology series, where everyday problems — spice racks, phone keypads, playlists — get connected to the algorithms and data structures that power real software. The goal is always the same: build the intuition first, then let the code fall out naturally.

Call To Action

If this walkthrough made the Frequency Ladder click for you, drop a like on the video and subscribe on YouTube so the next one reaches you automatically. For deeper dives and behind-the-scenes breakdowns, subscribe to the Substack newsletter. Comments with questions or alternate approaches are always welcome — and if you know someone prepping for interviews, send this their way.

Quick Review

What pattern solves "Minimum Pushes to Type Word II"?

Greedy frequency-to-slot assignment: sort letters by frequency, assign most frequent to fewest key-presses (8 keys $\times$ up to 3 letters).

When do you reach for greedy frequency-slot assignment?

When items map many-to-few buckets and cost depends on bucket depth — assign highest-frequency items to shallowest slots first.

Time complexity of the T9 greedy solution?

$O(n \log n)$ for sorting letter frequencies, where n is number of distinct letters (max 26).

Space complexity of the greedy T9 solution?

$O(1)$ extra beyond frequency array — at most 26 counts and 8 keys, both fixed-size.

Trickiest edge case in this problem?

Fewer than 8 distinct letters (e.g. 5 letters, 5 pushes) — every key holds one letter, so cost is just count of distinct letters, no depth multiplier needed.

How does this differ from simple frequency counting problems (e.g. Huffman)?

Huffman builds a variable-depth tree from scratch; here depth per slot is fixed (1,2,3) by the 8-key layout, so no tree-building — just sort and bucket.

Minimum Number of Pushes to Type Word II —  Beginner (1/3) — free Python cheat sheet